School of Engineering and Computer Science
AIML320/AIML420 — Artificial Intelligence

Assignment 1: Supervised Learning Algorithms

15% of Final Mark — Due: 11:59pm, 26th March 2026, Thursday

Objectives

The goal of this assignment is to enhance your comprehension of fundamental concepts and algorithms in supervised learning, develop computer programs to execute these algorithms, utilize these implementations for classification tasks, and analyze the outcomes to draw meaningful conclusions. The assignment covers the following topics:

- Basic machine learning concepts/fundamentals/paradigm
- Nearest neighbour method for classification
- Decision tree for classification
- Ensemble learning for classification
- Data preparation
- Feature selection and feature extraction

IMPORTANT

All requirements specific to AIML420 students are marked as (**AIML420 mandatory**). AIML320 students may choose to complete the tasks marked as (AIML420 mandatory). Completing it correctly can earn up to 2 bonus marks. Bonus marks will only be added if the total mark (including bonus) does not exceed 100; any excess will be discarded.

General Guidances

- **Programming language.** For Part 1, you are encouraged to use **Python** as the programming language. However, it is acceptable to use other programming languages if you choose to. Make sure to include clear instructions in your README file. If your code cannot be executed or if the instructions are unclear, it will impact your grade.

  Part 2 requires the use of scikit-learn, and other libraries as needed (such as NumPy and Matplotlib). The code can be written in a .py script or a Jupyter Notebook (.ipynb) (need to be executable in either format).

- **Library usage.** There are two parts for this assignment. Be mindful of when it is appropriate to use libraries like scikit-learn and when they must NOT be used, as they are required for different objectives and practices throughout the assignment. Basically, **Part 1 must be implemented from scratch** for the **core** decision tree algorithm, *directly using the built-in decision tree function in libraries for these programming will not be awarded any credit.* For example, no marks will be awarded if you directly using decision tree function that embedded in the scikit-learn libraries including sklearn.tree. However, libraries for data loading and visualization can still be used (such as pandas and matplotlib). In contrast, Part 2 requires the use of scikit-learn to implement the learning algorithms.

- **AI use.** Don't use AI to generate complete code solutions. That's not helpful for your learning. If you do generate part of the code with AI, you should read it carefully and make sure you understand it. You should also rigorously test the code to ensure it works as expected. In your report, you should disclose any uses of AI you made. We reserve the right to ask you questions if we have any concerns.

- **Data** There are two separate datasets: one for Part 1 and another for Part 2.
  - Part 1: For dataset for Part 1, the last column represents the class label. All features are **nominal**, represented as integers from 0 to $m-1$, where m is the total number of possible nominal values for the feature. These parts do not need to handle numerical attributes.
  - Part 2: For dataset for Part 2, the dataset is from the real world and involves classifying tumors as malignant (cancerous) or benign (non-cancerous). Data Preprocessing techniques such as cleaning and standardization of the different attributes are necessary.
- **Metric**. In Part 1 of this assignment, the only metric to be used is **overall classification accuracy**. In Part 2, additional metrics are required, i.e. **Precision, Recall, and F1 score**. All metrics can be either implemented from scratch or calculated using external libraries (e.g. sklearn).

Part 1: Decision Tree Method (50 marks)

Data

The datasets (rtg_A, rtg_B, and rtg_C) used for this section of the assignment were synthetically generated. The three datasets contains only nominal attributes (some are binary, some have more than 2 possible values) and 2 possible class labels. For this part of the assignment, **only the training datasets are provided**. The focus is not on evaluating accuracy on a test set but rather on analyzing the generated tree.
IMPORTANT: rtg_C is only required for AIML420 and is used exclusively in the "AIML420 mandatory" requirements.

Requirements

You are tasked with implementing a simplified version of the decision tree algorithm, using entropy-based Information Gain and an iterative data splitting strategy as discussed in our lecture. You may refer to the code template provided for programming, but it is not required to do so.

INPUT: Your program should take two arguments as command line arguments: file name for training data [1], and file name for the output (tree).

Example: **DecisionTree.py train.csv output_tree.txt**

OUTPUT: Print the accuracy of the training dataset based on the generated decision tree, and save the tree's information to output_tree.txt, including the following essential details:

- For split nodes: specify the features used for splitting, the Information Gain (IG), and the entropy.
- For leaves: specify the class counters associated with each leaf node.
- For edges: identify the values associated with the edges between nodes.

Note that the most straightforward method for outputting the tree's information involves performing a depth-first traversal of the tree structure.

Example tree output

Example shown in Figure 1:

In this example, there are two split nodes and three leaves. Each split node must indicate the feature it is splitting on. In our case, the first split occurs on "feature 1" while the second split occurs on "feature 2". Additionally, split nodes should provide information on the information gain (IG) and entropy of that node.

The leaf nodes contain counters for the instances reaching them based on the class label. For instance, leaf 0: 31, 1: 0 indicates that 31 instances reaching that node belong to class 0, while no instances of class 1 reach that node.

[1] Why isn't there a test set? This part of the assignment focuses on understanding the structure of the decision tree rather than evaluating its accuracy on a separate test set.

```
feature 1 (IG: 0.3008, Entropy: 0.6801)
-- feature 1 == 0 --
      leaf {0: 31, 1: 0}
-- feature 1 == 1 --
     feature 2 (IG: 0.9980, Entropy: 0.9980)
     -- feature 2 == 0 --
           leaf {0:0, 1: 9}
     -- feature 2 == 1 --
           leaf {0: 10, 1: 0}
```

Figure 1: Example output tree (text)

It's essential for the edges to specify the value of the feature corresponding to that branch. For instance, "– feature 1 == 0 –" signifies that the node below is reached when feature 1 equals 0. Another view of the same example is shown in Figure 2, **you don't need to provide these visualizations (the text version is enough)**.

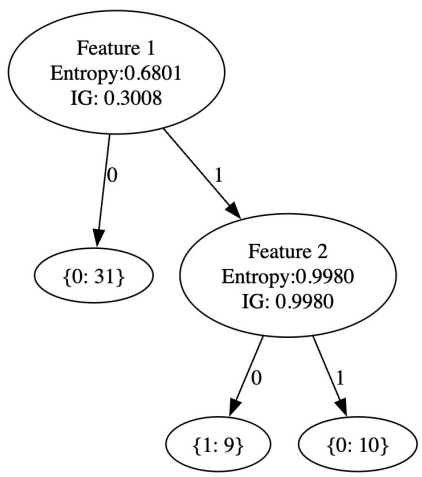

Figure 2: Example output tree (graph) - leaf counters for classes with 0 instances were omitted.

Impurity measure and stopping criteria

You should use the **Entropy impurity measure and Information Gain** presented in the lectures. To determine when to stop growing the tree, use a fixed threshold of 0.00001 for the Information Gain (i.e., if the IG from a split is less than 0.00001, do not split). Notice that if there are no more instances to split on, no further splits will take place as well.

Submissions

1. (25 marks) **Program code** for your decision tree classifier (the source code that can run on the ECS School machines).
2. **README** describing how to run your program.
3. (25 marks) **A report in .pdf format** and output file of tree's information. The report should include the answers to the questions below:
   (a) Report the overall classification accuracy for "rtg_A" and the decision tree classification results for each data instance. Generate another output file containing the information of the constructed decision tree. (suggested name: output_tree_A.txt).

(b) Report the overall classification accuracy for "rtg_B" and the decision tree classification results for each data instance. Generate another output file containing the information of the constructed decision tree. (suggested name: output_tree_B.txt).

(c) When analyzing the results for rtg_A and rtg_B, consider the following questions:

   i. Which feature results in the minimal entropy when considering the entire training dataset? How can this observation be made?

   ii. Were there any features that were not utilized in constructing the tree for both datasets? If so, what factors contributed to their exclusion from the tree-building process? Tip: you might want to observe the data to answer this question.

(d) **(AIML420 mandatory)** Report the overall classification accuracy for rtg_C and the decision tree classification results for each data instance.

(e) **(AIML420 mandatory)** When analyzing the results for rtg_C, consider the following questions:

   i. **(AIML420 mandatory)** Is the generated tree "good" for making predictions on unseen data? Justify your question based on the tree structure.

   ii. **(AIML420 mandatory)** Were there any features that were not utilized in constructing the tree for this dataset? If so, what factors contributed to their exclusion from the tree-building process? Tip: you might want to observe the data to answer this question.

   iii. **(AIML420 mandatory)** Are there any preprocessing steps that you would consider to potentially enhance the generalization ability of the generated tree? Note that while we lack a test set, you can draw conclusions based on the tree itself and how features were employed by the tree.

Part 2: Exploring Classification Algorithms (50 marks)

In this part of the assignment, you will explore six classifiers using scikit-learn. Instead of using your own implementation, you are required to use scikit-learn's built-in functions (including the Decision Tree classifier). The objective is to examine how data preprocessing and tuning key hyperparameters impact classification performance on the provided dataset (breast-cancer.csv). Each classifier will be tested with a range of plausible hyperparameter values, and its performance will be evaluated on a held-out test set. For consistency, please apply a fixed 70:30 train-test split using scikit-learn's `train_test_split()` function for all classification tasks.

Data

Breast cancer is the most common cancer worldwide. One of the main challenges in breast cancer detection is classifying tumors as either malignant (cancerous) or benign (non-cancerous). We provide a dataset (attached as "breast-cancer.csv") that can be used to build classification models to predict the cancer type .

The dataset consists of two categories (M for Malignant, B for Benign) and includes 32 attributes such as the radius of lobes, mean surface texture, and more. All attributes are numerical values with varying data ranges. Some values for certain attributes are missing, and these missing values are represented as 0 in the CSV file (Note: sometimes 0 is a valid value, but in some cases (like in the assignment), it is not so it can be used as a missing data indicator). When processing the missing data, you need to replace all 0 values with meaningful numerical values. There are a total of 569 instances in the dataset. You need to split the dataset into training dataset and testing dataset as suggested.

Task Description

Different classification methods have different characteristics when they interact with the data. Your goal is to implement the whole pipeline including data loading, cleaning, normalization, machine learning algorithms exploration and evaluation. **All results below are required to evaluate using accuracy, precision, recall, F1 score.** The tasks including:

1. Missing Data Processing and Data Normalization (10 marks)

(a) Implement two missing-data handling strategies (i.e., mean and median imputation) to the provided dataset (either using the scikit-learn library or from scratch). Evaluate their impact on the accuracy of a Decision Tree classifier with a plausible hyperparameter configuration. **Once the dataset is complete, use the full dataset for the following tasks.**

(b) Perform standardization and min-max scaling to normalize the data. Then, compare the performance of KNN and Decision Tree models with and without these two normalizations. Analyze whether these normalization techniques impact KNN and Decision Tree models to the same extent. You may select only one reasonable set of hyperparameters for each classifier for this comparison and keep it fixed.

2. Classifier Exploration and Hyperparameter Tuning (20 marks for AIML320 and 15 marks for AIML 420):

   Experiment with each classifier listed below using the scikit-learn library, adjust key hyperparameters for each, and evaluate their performance on the test set. The results should be presented in a table, as shown in Table 1. Observe and discuss the impact of varying these hyperparameters on the performance of each model. Additionally, compare and contrast the performance of the different classifiers (e.g., discuss which algorithms performed the best and provide reasons for their superior performance).

   (a) **K-Nearest Neighbors (KNN):** Experiment with number of neighbors`n_neighbors` values: 3, 9, 15, and 21.

   (b) **Decision Tree:** Test different values for depth of the tree `max_depth`: 2, 8, and 14.

   (c) **AdaBoost:** Adjust `n_estimators` to vary the number of weak learners (boosting rounds) with values [10, 20, 30].

   (d) **Random Forest:** Vary the number of trees `n_estimators`: 10, 30, 50, and 60.

Classifier	**Hyperparameter**	**Value**	**Accuracy (%)**	**Precision**	**Recall**	**F1 Score**
KNN	*n_neighbors*	3	?	?	?	?
		9	?	?	?	?
		15	?	?	?	?
		21	?	?	?	?
Decision Tree	*max_depth*	2	?	?	?	?
		8	?	?	?	?
		14	?	?	?	?
AdaBoost	*n_estimators*	10	?	?	?	?
		20	?	?	?	?
		30	?	?	?	?
Random Forest	*n_estimators*	10	?	?	?	?
		30	?	?	?	?
		50	?	?	?	?
		60	?	?	?	?

Table 1: Performance of Different Classifiers with Varying Hyperparameters

3. Feature Engineering and Dimensionality Reduction: (20 marks for AIML320 and 15 marks for AIML 420)

   (a) **Feature Selection using Pearson Correlation:** Select features based on Pearson correlation by applying a threshold of 0.6. Only features with a correlation coefficient greater than 0.6 with the target variable are retained. Train a Decision Tree model on the reduced feature set. State the major steps. Report the correlation matrix and the number of features retained. Evaluate classification performance before and after feature selection and discuss the results. An example of a correlation matrix is shown below Fig. 3. **You do not need to pay attention to the color or the order of the different attributes visualized here; a table containing only the correlation values on your dataset is sufficient for the assignment**.

(b) **Feature Extraction using PCA:** Implement Principal Component Analysis (PCA) for dimensionality reduction using scikit-learn library and evaluate its impact on the performance of **Decision Tree** classifiers. Compare the model performance with and without PCA on a plausible hyperparameter setting (e.g., depth of the tree `max_depth` equal to 2 and 8). Analyze and interpret the results. Considering the potential need for data normalization before applying PCA. Also, consider how to handle the test data when applying the transformation.

4. **(AIML420 mandatory)** Comparison: PCA vs. Feature Selection (5 marks)

   - Match the number of PCA components with the dimensions obtained from feature selection. Train a Decision Tree model using both PCA-transformed data and feature-selected data. Compare the results between both approaches and discuss the benefits and drawbacks of feature selection vs. PCA in terms of model performance and interpretability.

5. **(AIML420 mandatory)** k-Fold Cross-Validation (5 marks)

   - Implement 10-fold cross-validation for KNN (with hyperparameters k=3 and k=9) and Decision Tree (with hyperparameter `max_depth` =2 and 8). You can either implement it from scratch or use external libraries (e.g., scikit-learn). State the major steps. Report mean and standard deviation of accuracy and F1 score on different hyperparamter setups. The results should be presented in two separate tables, as shown in Table 2: one table for accuracy and another table for the F1 score. Discuss model stability and reliability based on these results.

	KNN (k=3)	**KNN (k=9)**	**Decision Tree (max_depth=2)**	**Decision Tree (max_depth=8)**
Fold 1	?	?	?	?
Fold 2	?	?	?	?
Fold 3	?	?	?	?
Fold 4	?	?	?	?
Fold 5	?	?	?	?
Fold 6	?	?	?	?
Fold 7	?	?	?	?
Fold 8	?	?	?	?
Fold 9	?	?	?	?
Fold 10	?	?	?	?
Mean	?	?	?	?
Std Dev	?	?	?	?

Table 2: 10-Fold Cross-Validation Results on one metrics (accuracy or F1 score) for KNN and Decision Tree Classifiers with Different Hyperparameter Values

Submission Guidelines

Submission Requirements

1. To avoid confusion, the programs for each part should be stored in separate directories **part1/** and **part2/**. In each directory, please provide a README file that specifies how to run (and compile) your programs on the ECS School machines. If the markers can't run your code, you will lose marks. Add necessary comments in your program, and make sure to include ALL the required libraries and dependent files.

2. **readme.txt**: The **readme file** presents the detailed steps and commands on how to run your programs using the command line. If your programs cannot run properly, then you should provide the **list of bugs** in this file. On the other hand, if your program can run properly, then you should include the **sample terminal output** of your program in this file.

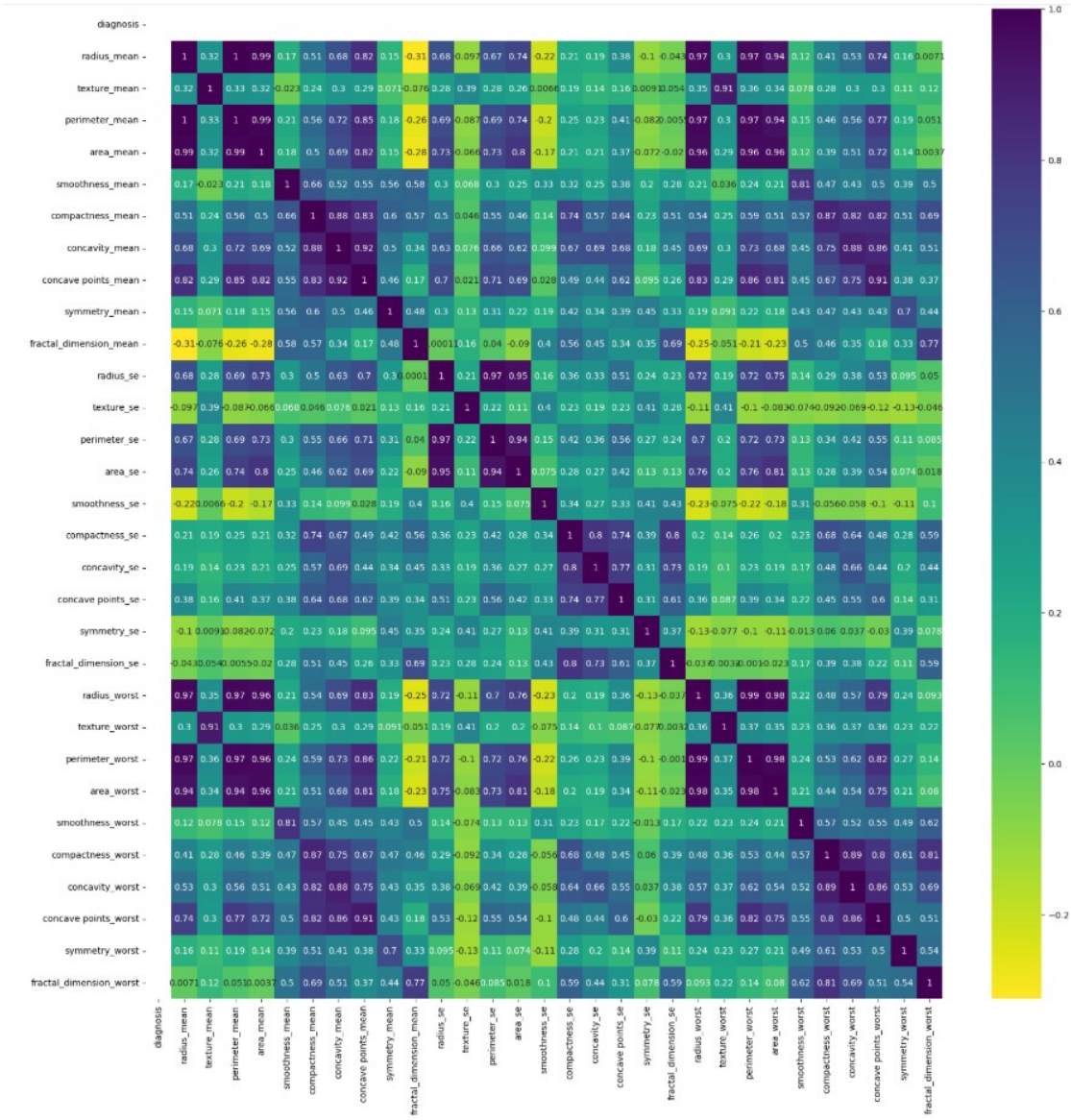

Figure 3: Example of correlation matrix.

3. You should use **a single PDF** report for all parts, and label questions clearly. This document allows you to practice your technical writing skills. It should contain clear answers to all questions of this assignment. In general, **readers should be able to understand what you have done purely from the report, without looking at other sources of information (e.g., source code) provided by you.**

Submission Method

The PDF version of the assignment report and your programs and readme files should be submitted electronically through the Web submission system (accessible from the AIML320 or AIML420 course website) by the due date.

Late Penalties

The assignment must be submitted on time unless you have made a prior arrangement with the **course coordinator** or have a valid medical excuse. We use the ECS extension system for all extension requests. Please make a request there if you think you have a valid reason.

The penalty for assignments that are handed in late without prior arrangement is one grade reduction per day. Assignments that are more than one week late will not be marked.
Remember that you have three late days for this course (which can be used fractionally), but these apply across the whole course, not per-assignment. Please save some late days in case you need them later on.

Plagiarism

Plagiarism in programming (copying someone else's code) is just as serious as written plagiarism, and is treated accordingly. Make sure you explicitly write down where you got code from (and how much of it) if you use any other resources asides from the course material. Relying heavily on borrowed code may lead to deductions in marks, whereas plagiarism warrants a zero mark and may entail disciplinary actions.